Uber :

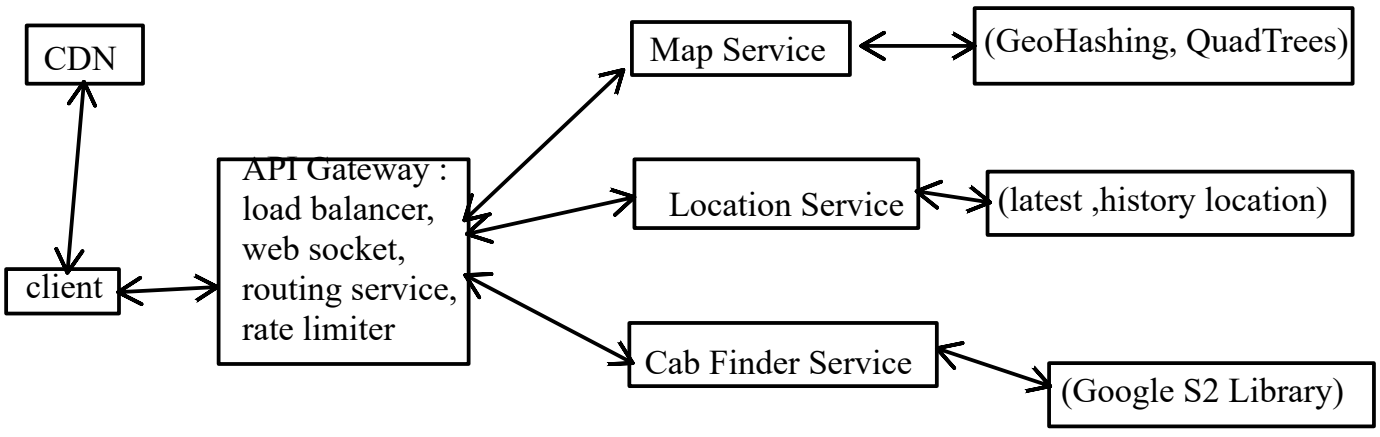
Functional requirements:
user should be able to book a cab based on his preference,
user should be able to search for any location based on his preference,
rider should be able to accept/reject any ride based on his preference.

Non-functional requirements:
data-partitioning,
load balancing,
fault tolerance,
vertical scalability,
horizontal scalability,
low latency,
high throughput,
CAP theorem (Here we need to prioritise Strong Consistency for booking scenario in order to avoid double booking/double payment & for search scenraio we need to prioritise 99.99% i.e Availability>>Consistency)

Core Entities:
user,
booking,
payment,
history

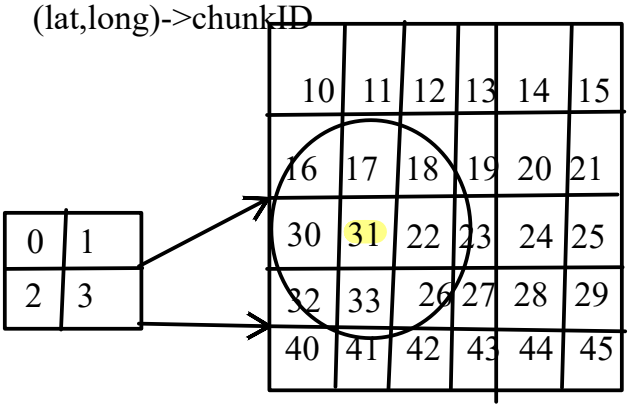
API :
POST:booking
{
userID,
{filemetadata}
}
GET:/search/{locationID}

HLD :

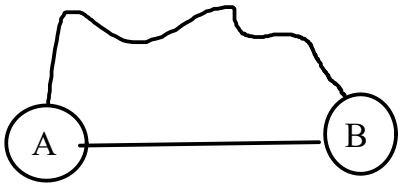


GeoHashing

(lat,long)->chunkID



Google S2 Library



Consistent hashing , gossip protocol